

A Lean Quantum Document Pipeline

Hybrid Quantum Machine Learning and Quantum String Algorithms
for Document Extraction, Benchmarked Against Classical Baselines

Pushkar Kumar

QIntern 2026 — QWorld

Mentor: Potluri Krishna Priyatham

August 2026

Repository: https://github.com/Pushkar0997/qi26_25

Live demo: https://pushkar0997.github.io/qi26_25/

Abstract

Document intelligence pipelines built on large language models carry substantial computational cost, motivating interest in whether quantum components can deliver comparable capability at reduced resource requirements. This project implements an end-to-end document extraction pipeline in which two of nine stages are quantum: a quanvolutional feature extractor operating on character crops, and a Grover-based comparator oracle performing pattern search over the extracted identifier field. Both are benchmarked against dimension-matched classical equivalents on a controlled synthetic corpus of 60 documents spanning five degradation tiers.

The central results are negative and are reported as such. Measured across 30 paired train-test splits, the quanvolutional feature extractor is statistically indistinguishable from a dimension-matched random classical convolution ($p \approx 0.2$ – 0.3 , winning approximately half of splits), and both are outperformed by raw pixel values at lower dimensionality ($p < 10^{-10}$, 30 of 30 splits). Optimising the filter’s rotation angles does not close this gap. For the search stage, a genuine in-circuit comparator oracle is constructed and verified with zero uncomputation leakage; however, measured two-qubit gate count scales as $N^{1.39}$ with text length, so the $O(\sqrt{N})$ query advantage of Grover’s algorithm does not survive the cost of loading classical text into the oracle. Against Tesseract on identical images, the pipeline is three to four times worse on clean input.

Two methodological findings proved more consequential than any quantum result. A train/serve distribution mismatch between training and inference crops inflated document character error rate by a factor of 6.5; correcting it was the single largest improvement in the project. Separately, isolated-character accuracy proved an actively misleading proxy for end-to-end accuracy, with a higher-scoring configuration producing substantially worse documents. The contribution of this work is therefore a measured characterisation of why the quantum components did not help at this scale, supported by a fully reproducible artifact.

Contents

1	Introduction	2
1.1	Motivation	2
1.2	Scope and Honest Framing	2
1.3	Contributions	2

2	Background	3
2.1	Quantum Image Encoding	3
2.2	Quantumvolutional Layers	3
2.3	Grover Search and String Matching	3
3	Pipeline Architecture	3
3.1	Dataset	4
4	Track A: Quantum Feature Extraction	4
4.1	Encoding Comparison: FRQI versus NEQR	4
4.2	The Quantumvolutional Filter	5
4.2.1	Efficient Simulation	5
4.2.2	Readout Design	5
5	Track B: Quantum String Matching	5
5.1	A Construction That Does Not Search	5
5.2	An In-Circuit Comparator Oracle	6
5.3	Staged Verification	6
5.4	Resource Scaling	6
6	Benchmarking	7
6.1	Experimental Design	7
6.2	Statistical Significance	7
6.3	Interpretation	8
6.4	Variational Filter	8
6.5	Shot-Noise Sensitivity	9
7	End-to-End Results	9
7.1	Train/Serve Skew: The Largest Single Error Source	9
7.2	External Baseline	10
7.3	Where the Remaining Error Lies	11
8	Noise and Hardware Feasibility	11
9	Reproducibility	11
10	Conclusions	12
10.1	Future Work	13
	Scope and Contributions	13

1 Introduction

1.1 Motivation

Extracting structured information from document images—invoices, forms, archival records—is a mature application of classical computer vision and optical character recognition (OCR). Contemporary approaches increasingly route this task through large language models, which achieve strong accuracy at considerable computational expense. The QIntern 2026 project brief posed the question of whether quantum machine learning and quantum string algorithms could deliver a “lean” alternative: comparable extraction capability at reduced resource cost.

This question decomposes naturally into two independent sub-problems, which the brief assigned to separate tracks:

- **Track A (vision).** Can a quantum feature extractor—specifically a quantum convolutional layer, the quantum analogue of a convolution—provide useful features for character recognition?
- **Track B (search).** Can Grover’s algorithm accelerate pattern search over extracted text, exploiting its $O(\sqrt{N})$ query complexity?

Both are legitimate questions with published affirmative-leaning literature, and both are answered negatively here, for reasons that are measured rather than asserted.

1.2 Scope and Honest Framing

This work makes no claim of quantum advantage. The explicit goal, following the project brief, was to construct a working pipeline and characterise its resource requirements against classical baselines. Where the quantum components underperform, that is reported directly, together with the mechanism responsible.

Three constraints bound what can be concluded. First, all quantum execution is *simulated*; no physical quantum hardware was available. Second, the corpus is synthetic, which is necessary for exact ground truth but does not represent real scanned documents. Third, the scale is small—4 qubits, 8×8 crops, a 36-class alphabet—so conclusions apply to this regime and should not be extrapolated.

1.3 Contributions

1. A complete nine-stage pipeline in which quantum feature extraction and quantum search are integrated with classical segmentation and classification (Section 3).
2. A comparator-based Grover oracle that performs string comparison *inside* the circuit, replacing a common construction that computes match positions classically before circuit construction (Section 5).
3. Statistical evaluation of the quantum-versus-classical feature comparison across 30 paired splits, distinguishing a null result from a real effect (Section 6).
4. Measured resource scaling for the Grover oracle, demonstrating that data loading dominates the quadratic query advantage (Section 5.4).
5. An external baseline against Tesseract, and identification of the classical segmentation front end—not either quantum stage—as the dominant error source (Section 7).
6. A fully reproducible artifact: committed dataset, committed model, continuous integration that fails if the report and the measured results disagree (Section 9).

2 Background

2.1 Quantum Image Encoding

Before any quantum processing can occur, pixel values must become quantum states. Two standard encodings were evaluated.

FRQI (Flexible Representation of Quantum Images) stores intensity as a rotation angle on a single shared colour qubit:

$$|I\rangle = \frac{1}{2^n} \sum_{i=0}^{2^{2n}-1} (\cos \theta_i |0\rangle + \sin \theta_i |1\rangle) \otimes |i\rangle \quad (1)$$

One qubit carries all colour information regardless of image size, but intensity is recovered only statistically: θ must be estimated from measurement frequencies, so reconstruction is approximate and shot-limited.

NEQR (Novel Enhanced Quantum Representation) stores intensity as a bit string on a colour register:

$$|I\rangle = \frac{1}{2^n} \sum_i |f(i)\rangle \otimes |i\rangle \quad (2)$$

An 8-bit greyscale image requires 8 colour qubits, but reconstruction is exact.

2.2 Quanvolutional Layers

Introduced by Henderson et al., a quanvolutional layer is the quantum analogue of a convolution: a small window slides over the image and, at each position, the patch is encoded into a quantum state, transformed by a circuit, and measured. The canonical formulation uses a *fixed random* circuit—the filter is not trained—which is the configuration evaluated here, with a trained variational variant examined separately in Section 6.4.

2.3 Grover Search and String Matching

Grover’s algorithm locates a marked item among N in $O(\sqrt{N})$ oracle queries. Applying it to string matching promises quadratically faster pattern location. The critical requirement, examined in detail in Section 5, is that the oracle must *perform* the comparison rather than being supplied its result.

3 Pipeline Architecture

The pipeline comprises nine stages, of which stages 4–5 and 9 are quantum:

#	Stage	Description
1	preprocess	median filter, Otsu threshold, speckle removal
2	segment	projection profiles $\rightarrow$ character boxes
3	crop	pad to square, resize to 8×8 , normalise
4	encode	R_Y angles $\rightarrow$ 4 qubits per patch (quantum)
5	quanvolutional	entangling circuit $\rightarrow$ 160 features (quantum)
6	classify	multinomial logistic regression $\rightarrow$ characters
7	assemble	rebuild lines, insert spaces
8	locate	identify the document identifier field
9	Grover search	comparator oracle $\rightarrow$ pattern position (quantum)

3.1 Dataset

Sixty documents were generated across five degradation tiers—clean digital, clean scan, noisy scan, clear handwriting, degraded handwriting—yielding **6,023 labelled character crops** over 36 classes. Generation is deterministic under seed 26.

Synthetic generation was chosen for two reasons. Character error rate requires exact ground truth, and hand-labelling scraped documents was not feasible within the project timeframe. More importantly, every tier renders the *same distribution of source text*, so any accuracy difference between tiers is attributable to image degradation alone rather than to differing content difficulty.

The degradation is a controlled variable: blur, sensor noise, contrast reduction, low-DPI resampling, rotation and blotch artifacts, with per-tier parameters recorded in the generator. That the tiers form a genuine difficulty gradient is confirmed by median per-crop dynamic range, which falls monotonically from 197 (clean digital) to 62 (degraded handwriting).

Limitations of the corpus. The handwriting tiers use an italic serif face with per-glyph position, rotation and scale jitter as a proxy for handwriting variability. This is not handwriting, and the IAM Handwriting Database is identified as the correct extension. This limitation is stated in the generator source, the repository README and here.

4 Track A: Quantum Feature Extraction

4.1 Encoding Comparison: FRQI versus NEQR

Both encodings were implemented, verified by reconstruction, and measured across three patch sizes.

Table 1: Encoding resource comparison. Gate counts are measured after transpilation to a $\{u, CX\}$ basis.

Patch	FRQI qubits	NEQR qubits	FRQI CX	NEQR CX	Qubit ratio
2×2	3	10	32	114	$3.33\times$
4×4	5	12	384	1,558	$2.40\times$
8×8	7	14	3,584	10,998	$2.00\times$

Verification confirmed the expected behaviour: NEQR reconstructs pixel values exactly, while FRQI reconstructs to within 1–2 intensity levels out of 255 at 8,192 shots.

A refuted prediction. An early technical summary written during this project predicted that the qubit gap between the two encodings would *widen* with patch size. Measurement refutes this: the ratio *narrows*, from $3.33\times$ to $2.00\times$. The original reasoning conflated two quantities. FRQI’s colour register is fixed at 1 qubit and NEQR’s at 8, so the colour gap is a *constant* 7 qubits at every patch size, while both encodings share an identically growing position register of $\lceil \log_2(\text{pixels}) \rceil$. A constant gap divided by a growing shared base necessarily shrinks as a ratio.

Absolute CX counts grow steeply for both, with NEQR reaching approximately 11,000 at 8×8 . It is specifically the *relative* qubit penalty that decays. The pipeline uses neither encoding directly—a patch-based quancvolutional filter does not require position addressing—but these measurements establish the resource baseline for whole-image quantum representation, and the 8×8 NEQR figure is the quantitative reason the pipeline is patch-based.

4.2 The Quanvolutional Filter

The filter is a fixed 4-qubit circuit: two layers of R_Y rotations with a ring of CX gates between them. Rotation angles are drawn once from a seeded generator and held constant.

Each 8×8 character crop is divided into sixteen 2×2 patches. Each pixel value v becomes a rotation angle $\theta = \pi v / 255$ applied to its own qubit, so a dark pixel remains near $|0\rangle$ and a bright one rotates toward $|1\rangle$. Because each qubit is rotated independently, the input is a *product state*; all entanglement in the output originates in the filter’s CX gates.

4.2.1 Efficient Simulation

A naive implementation submits one simulator job per patch, requiring approximately 96,000 jobs for a single pass over the dataset—several hours of wall-clock time, which makes iteration impractical.

Because the filter is fixed, its unitary U is identical for every patch. It is extracted once and applied as a batched matrix multiplication:

$$|\psi_{\text{out}}\rangle = U |\psi_{\text{in}}\rangle \quad (3)$$

This is not an approximation but the same linear algebra the simulator performs, without per-job overhead and without shot noise. Agreement with the shot-based implementation was verified at a maximum deviation of **0.0028** across random patches at 200,000 shots—within sampling error. Processing time fell to 0.15 seconds for 2,000 characters.

4.2.2 Readout Design

The obvious readout is per-qubit $P(\text{measure } 1)$: four values. However, a 4-qubit circuit produces a distribution over 16 outcomes, and retaining only four marginals discards every correlation the entangling layer created—precisely the component a classical product-state model could not produce.

Adding pairwise correlations $\langle Z_i Z_j \rangle$ recovers six additional values per patch, giving 160 features per character rather than 64.

Table 2: Effect of readout design, all tiers pooled.

Readout	Feature dimension	Accuracy
Marginals only	64	86.7%
Marginals + $\langle Z_i Z_j \rangle$	160	92.2%

The 5.5-point gain from readout design alone exceeds any difference measured between the quantum and classical filters, which is itself a result: for quanvolutional approaches, how much of the output distribution is retained matters more than which circuit produced it.

5 Track B: Quantum String Matching

5.1 A Construction That Does Not Search

A common formulation of Grover-based string matching, and the one present in this project’s own starter notebook, proceeds as follows:

```
match_positions = [i for i in range(len(text) - len(pattern) + 1)
                    if text[i:i+len(pattern)] == pattern]
oracle = build_marked_state_oracle(n_qubits, match_positions)
```

The comparison executes in Python, before any qubit is allocated. The circuit is then constructed to flip the phase of positions already known. This demonstrates Grover *amplification* correctly—the measured distribution peaks at the correct position—but it is not string matching, and no complexity claim can rest on it, because the $O(N)$ classical scan has already occurred.

5.2 An In-Circuit Comparator Oracle

The replacement receives only the text and the pattern. Two registers are used: a position register `pos` of $\lceil \log_2(N - M + 1) \rceil$ qubits holding candidate start positions in superposition, and a window register `win` of $M \times b$ qubits, where b is the alphabet width in bits.

Four stages execute:

1. **Window loading.** For each concrete position i , multi-controlled writes load the bits of `text[i : i + M]` into `win`, conditioned on `pos = i`. Afterwards $|i\rangle|0\rangle \rightarrow |i\rangle|\text{text}[i : i + M]\rangle$.
2. **Pattern XOR.** An X gate is applied to `win[j]` wherever pattern bit j is 1. The register is now all-zeros exactly when the window equals the pattern.
3. **Phase marking.** A multi-controlled phase gate, sandwiched between X layers, flips the phase iff `win` is all-zeros.
4. **Uncomputation.** Stages 2 and 1 are reversed, disentangling `win` and returning it to $|0\rangle$.

Stage 4 is not housekeeping. Without it, the window register remains entangled with the position register and functions as a which-path record: the amplitudes cannot interfere and the diffusion operator amplifies nothing.

An encoding detail. Alphabet codes begin at 1, never 0. When $N - M + 1$ is not a power of two, the position register addresses more states than there are valid positions; those unreachable states never trigger a controlled load, so their window remains all-zeros—exactly the pattern stage 3 marks as a match. Reserving code 0 eliminates this failure mode.

5.3 Staged Verification

Correctness was established in stages rather than by observing a plausible final output.

Table 3: Oracle verification. Leakage is total probability outside the `win = 0` subspace after uncomputation.

Stage	Check	Result
1	Window register holds <code>text[i : i + M]</code> for a probe position	pass
2	Exactly the true match positions have negative amplitude	pass
3	Uncomputation leakage	0.00
4	Full search returns a true match	pass, 95–96% confidence

On a 16-character hexadecimal field with a 2-character pattern: 14 qubits, 3 Grover iterations, correct position recovered in 95.2% of shots.

5.4 Resource Scaling

Grover’s algorithm provides $O(\sqrt{N})$ queries. However, the window loading stage must write the text into the circuit, and each candidate position requires its own controlled load.

Fitting a power law in log–log space gives a CX scaling exponent of **1.39**: *superlinear* in text length. The measured curve lies above a linear reference, nowhere near $\sqrt{N}$.

Table 4: Measured oracle resources against text length, hexadecimal alphabet, 2-character pattern.

N (characters)	Qubits	Depth	CX gates
8	13	1,857	1,082
12	14	4,583	2,492
16	14	6,383	3,428
24	15	11,445	6,122
32	15	16,159	8,570
48	16	29,664	15,696
64	16	39,558	20,904

Interpretation. For searching stored, unstructured classical text that must be loaded into the circuit, the quadratic query advantage does not translate into an end-to-end speedup; the loading cost dominates. This is a well-known caveat in the literature, and the contribution here is reproducing it with direct measurement rather than quoting $\sqrt{N}$ unqualified.

Grover’s advantage remains real where the text is already available as a quantum oracle or in QRAM; where the search space is *generated* by a function rather than stored, as in satisfiability or cryptographic preimage search; or where a loaded state is reused across many queries, amortising the cost. None of these describes scanning a document just received.

A second practical constraint: a full 38-symbol alphabet requires 6 qubits per character, placing even a short window beyond feasible simulation. The pipeline therefore searches the hexadecimal-representable identifier field, which is the realistic target for structured-field lookup in any case.

6 Benchmarking

6.1 Experimental Design

All feature extractors were compared *at matched dimensionality*, on identical crops, with an identical classifier and split. Without matching dimensionality, a win would be indistinguishable from granting one method a wider representation.

Table 5: Character accuracy by extractor and tier. Dimensions in parentheses.

Tier	quanv (64)	quanv+ZZ (160)	classical (160)	raw pixels (64)
clean_digital	99.3%	100%	100%	100%
clean_scan	99.3%	100%	100%	100%
noisy_scan	93.0%	97.7%	97.7%	98.3%
clear_handwriting	87.7%	96.7%	96.3%	98.7%
degraded_handwriting	62.5%	75.7%	74.4%	79.4%
All tiers	86.7%	92.2%	91.9%	93.6%

6.2 Statistical Significance

A single split yields point estimates but cannot distinguish a real difference from split-to-split noise. The comparison was repeated over 30 splits, with every extractor scored on the same splits so that comparisons are paired.

This separates two claims that a single-split table presents identically. The quantum-versus-classical gap is noise: the quantum layer wins approximately half of the splits. The raw-pixel

Table 6: Paired comparisons across 30 splits. Figures are given as ranges because scikit-learn’s solver is not bit-reproducible across BLAS builds; conclusions are stable across runs, individual decimal places are not.

Comparison	Mean difference	p	Verdict
quanv+ZZ – classical	−0.1 pt	$\approx 0.2\text{--}0.3$	not distinguishable from noise
raw pixels – quanv+ZZ	+1.2 pt	$< 10^{-10}$	significant
raw pixels – classical	+1.1 pt	$< 10^{-12}$	significant
quanv+ZZ – quanv	+5.7 pt	$< 10^{-25}$	significant

lead is not: it holds in all 30.

6.3 Interpretation

Three findings, in order of importance.

At matched dimensionality, the quantum layer performs at parity with its classical analogue. It is not *specifically* worse; a random classical convolution with the same patch size, stride and output width scores the same within noise. Whatever limits performance is not quantumness.

Both lose to raw pixels, at lower dimensionality. This reframes the comparison: the relevant axis is not quantum versus classical but *untrained random feature map* versus *no feature map*. At 8×8 the input is already close to information-minimal, so any fixed random projection discards more than it contributes. That the classical control fails identically is what identifies the cause.

Readout design dominated filter design, as noted in Section 4.1.

6.4 Variational Filter

The project brief specified a *variational* circuit. The filter’s eight R_Y angles were optimised by gradient-free Powell search, starting from the untrained baseline’s exact angle vector. The classical control was trained identically—training only the quantum side would invert the fairness the untrained comparison was designed to preserve.

Table 7: Variational training, evaluated on a sealed test partition of 1,807 crops.

Configuration	Parameters	Accuracy	Δ
Quantum, untrained	—	91.7%	—
Quantum, trained	8	90.6%	−1.1 pt
Classical, untrained	—	92.0%	—
Classical, trained	40	92.1%	+0.1 pt
Raw pixels	—	93.6%	—

Training does not close the gap. The Section 6 conclusion therefore strengthens: the deficit is not caused by the filters being untrained, because training them does not help.

A methodological correction. An earlier version of this experiment reported a +1.4-point gain for the trained quantum filter, bringing it level with raw pixels. That result was an artifact. The angle search had been run over the full dataset, and the final evaluation then used a fresh random split of that same dataset, so the angles had been selected using rows that later appeared in the test set. Re-running with a sealed 30% partition the search never sees reduced the gain from +1.4 points to approximately zero. The inflated figure is recorded here rather than discarded,

because the difference between the two is the difference between a false headline claim and the actual finding.

One property does survive: the quantum filter produces its 160-dimensional feature space from 8 parameters against the classical control’s 40. At the untrained baseline the two perform equivalently, so that $5\times$ parameter economy is real and costs nothing there. It does not survive this training protocol, and should not be reported without that qualification.

6.5 Shot-Noise Sensitivity

Exact statevector results are achievable only in simulation; hardware samples.

Table 8: Accuracy against measurement shot budget per patch.

Shots	Accuracy
64	70.7%
256	81.7%
1,024	85.2%
4,096	86.7%
exact	86.7%

Approximately 1,024 shots per patch are required to approach the noiseless result. At 16 patches per character this is roughly 16,000 circuit executions per character—a substantial hardware cost that belongs in any honest resource accounting and is frequently omitted from published quanvolutional results.

7 End-to-End Results

Two measures are used. **Character error rate (CER)** is the edit distance between extracted and reference text, normalised by reference length. **Document ID recovered exactly** is the fraction of documents whose identifier field is completely correct—the pipeline’s actual task, for which a single wrong character renders the result useless.

Table 9: End-to-end performance, all 12 documents per tier.

Tier	CER	Segmentation ratio	ID recovered exactly
clean_digital	6.6%	0.95	100%
clean_scan	8.5%	0.93	58%
noisy_scan	79.5%	0.34	0%
clear_handwriting	51.8%	0.70	0%
degraded_handwriting	85.7%	0.99	0%

The two metrics diverge sharply. **clean_scan** reads at 8.5% CER—visually almost clean—yet only 58% of its documents yield a usable identifier. Averaged character accuracy flatters a system that fails the actual task in almost half of cases.

7.1 Train/Serve Skew: The Largest Single Error Source

The largest improvement in this project came from a data-handling defect rather than any modelling change.

Detection. Switching the feature stage from the 64-dimensional marginal readout to the 160-dimensional $+ZZ$ readout *raised* isolated character accuracy yet made end-to-end CER *worse*. A strictly better feature set producing strictly worse output is impossible if both are measured on the same input distribution—so they were not.

Cause. Training crops were cut from exact glyph bounds recorded during dataset generation; inference crops come from projection segmentation. These are measurably different distributions: on `clean_scan`, ground-truth boxes have median size 16×19 px against segmented boxes at 12×15 px. The higher-capacity features fitted the training distribution more tightly and transferred less well.

Correction. Training crops are now built by running the real segmentation path over training pages and labelling each detected box from ground truth by overlap, with ambiguous detections dropped rather than guessed.

Table 10: Effect of correcting train/serve skew.

Tier	CER before	CER after	Reduction
<code>clean_digital</code>	42.9%	6.6%	6.5×
<code>clean_scan</code>	29.7%	8.5%	3.5×
<code>clear_handwriting</code>	74.5%	51.8%	1.4×

A caution on the accuracy figure. Training on segmented crops *lowers* reported held-out character accuracy from 92.6% to 67.7%. These numbers are not comparable—they are measured on different crop distributions—and the lower one is measured on the harder, realistic one. The comparison to draw is that a model scoring 92.6% on clean bounds produced 42.9% document CER, while a model scoring 67.7% on realistic bounds produced 6.6%.

Isolated-character accuracy was an actively misleading proxy for end-to-end accuracy, and optimising it drove the pipeline in the wrong direction. Any comparison of feature extractors on pre-cut crops—including Section 6 of this report—measures something narrower than pipeline quality.

7.2 External Baseline

Sections 6 and 7 compare feature maps that all share this project’s segmentation front end and classifier. Tesseract provides an external reference, run on the same images over the same twelve documents per tier, scored with the same error function.

Table 11: Comparison against Tesseract.

Tier	This pipeline	Tesseract	Pipeline ID	Tesseract ID
<code>clean_digital</code>	6.6%	2.0%	100%	92%
<code>clean_scan</code>	8.5%	1.9%	58%	83%
<code>noisy_scan</code>	79.5%	100.0%	0%	0%
<code>clear_handwriting</code>	51.8%	20.2%	0%	17%
<code>degraded_handwriting</code>	85.7%	100.0%	0%	0%

Tesseract is three to four times better on clean and moderately degraded input. This is the expected result and is reported plainly: four decades of engineering on a task-specific problem outperforms a 4-qubit filter over 8×8 crops.

Two details complicate rather than rescue the picture. On `clean_digital` this pipeline recovers the identifier in every case against Tesseract’s 92%, because a constrained 36-class charset cannot emit the out-of-charset characters Tesseract occasionally produces. On the two most degraded tiers Tesseract returns essentially nothing (100% error) where this pipeline still recovers roughly one character in five—an artifact of Tesseract’s page-analysis stage rejecting input it judges unreadable, rather than evidence of a superior method.

The comparison is not controlled: Tesseract brings its own segmentation, language model and training corpus. It is included because the first question a sceptical reader asks is how the pipeline compares to an ordinary classical tool, and the answer belongs in the report regardless of whether it flatters the work.

7.3 Where the Remaining Error Lies

On clean tiers, segmentation recovers 93–95% of characters and isolated character accuracy approaches 99%, yet CER is 6.6–8.5%. The residual is therefore not classification but missed or merged glyphs and word-spacing reconstruction. **The classical front end, not either quantum stage, is the accuracy bottleneck.**

8 Noise and Hardware Feasibility

All results above assume noiseless simulation. Three degradation mechanisms were modelled to estimate hardware behaviour: shot noise (Section 6), gate error, and readout error.

Gate noise was modelled with a depolarising channel. Because the filter circuit is small, the noisy channel was constructed once as a 256×256 superoperator and applied by batched matrix multiplication, verified to reproduce the noiseless path exactly at $p = 0$.

A measurement subtlety. Accuracy barely moves under gate noise when the classifier is refitted at each noise level. This is misleading: depolarising noise contracts features uniformly toward the maximally mixed state, and refitting simply learns larger weights. A flat curve could therefore mean “the noise did not matter” *or* “the retraining absorbed it.” Reporting a train-clean/deploy-noisy transfer measurement alongside removes the ambiguity; the two agree below 3×10^{-2} error and diverge above it, which is what establishes the robustness as real rather than an artifact of retraining.

Table 12: Circuit size against error tolerance, at a two-qubit gate error rate of 10^{-3} .

Circuit	CX gates	$P(\text{error-free})$
Quantumvolutional filter, per patch	8	99.2%
Grover oracle, 16-character field	3,272	3.8%
Grover oracle, 3 iterations	9,816	5×10^{-5}

The conclusion is stark and is an independent argument from the data-loading result of Section 5.4: **the feature extractor is plausible on near-term hardware; the search stage is not.** The difference is three orders of magnitude in circuit size. The search stage requires error correction, not improved calibration.

9 Reproducibility

Every figure in this report regenerates from a clean repository clone. Three measures support this.

The dataset and trained model are committed. Both were initially gitignored as derived artifacts. This was incorrect: generation renders text with whatever fonts the host machine provides, and training is not bit-reproducible across BLAS builds. During the project, a regeneration on a machine whose Pillow was built with libraqm produced identical text with every glyph shifted by 0–2 pixels—enough to change clean-document CER from 6.6% to 63.7% in an earlier configuration. The generator now pins the text layout engine explicitly and the dataset manifest records the Pillow, FreeType and libraqm state.

The browser demonstration is verified against the Python implementation. An accompanying interactive demonstration runs the full pipeline client-side. It is not a re-implementation: it loads the exact exported weights, and a parity harness asserts stage-by-stage agreement across three quality tiers, with features matching to 5.6×10^{-16} and decoded text identical. Achieving this required matching Pillow’s antialiased triangle resampling filter, its integer truncation semantics, its 22-bit fixed-point accumulation, its integer luma conversion, and SciPy’s 4-connectivity default—each of which silently altered decoded characters when mismatched.

Continuous integration gates the claims. On every push, a workflow re-runs the benchmark against tolerance bands, executes the oracle self-test, verifies browser parity, checks that the significance conclusions still hold, and fails the build if any figure in the results file does not appear in this report. That final gate has caught a real contamination: a results file measured on a drifted dataset.

10 Conclusions

1. A complete hybrid pipeline was constructed and runs end to end: quantum feature extraction feeding a classical OCR backend, feeding a genuine Grover comparator search over the recovered identifier field.
2. At matched dimensionality the quantum feature layer is statistically indistinguishable from a classical convolution ($p \approx 0.2$ – 0.3 , approximately half of 30 splits), and both are outperformed by raw pixels ($p < 10^{-10}$, 30 of 30). On this task, at this scale, quantum feature extraction provides no accuracy advantage.
3. Grover string matching functions correctly, but measured gate cost grows superlinearly (exponent 1.39) with text length owing to data loading, so the $\sqrt{N}$ query advantage does not translate into an end-to-end speedup for stored classical text.
4. The dominant error source in the full pipeline is the classical segmentation front end, not either quantum stage. Clean-document CER is 6.6–8.5%, with 100% identifier recovery on clean digital documents.
5. The largest single improvement came from correcting a train/serve skew, not from any modelling change—and was found only because a better feature set produced worse end-to-end output.
6. An early prediction made within this project, that the FRQI/NEQR qubit gap would widen with patch size, is refuted by measurement; it narrows from $3.33\times$ to $2.00\times$.
7. Against Tesseract on identical images, this pipeline is three to four times worse on clean input. It is not competitive with mature classical OCR.

This work does not demonstrate quantum advantage for document intelligence, and identifies specific measured reasons why: untrained random feature maps underperform the identity at small input sizes, and oracle data loading erases Grover’s asymptotic edge. Both are useful negative results, and both are more informative than a tuned demonstration would have been.

10.1 Future Work

1. **A trained filter with a smooth objective.** The variational experiment optimised a step-function accuracy metric with a gradient-free method. Parameter-shift gradients on a smooth surrogate loss, with cross-validated folds, is the correct approach.
2. **Replacing the segmentation front end.** It is the measured bottleneck; a learned detector would likely improve end-to-end results more than any change to the quantum stages.
3. **Real handwriting.** The handwriting tiers are a font-plus-jitter proxy. The IAM Handwriting Database is the appropriate extension.
4. **Stem-level or block-level encodings for search.** The one-qubit-per-position encoding is what makes the oracle expensive; coarser encodings would reduce both qubit count and loading cost.
5. **Execution on physical hardware** for the feature extractor, which the noise analysis suggests is feasible.

Scope and Contributions

This project was scoped for three participants: Klasik Taidi on Track A (vision), Hoang Dinh Duy Anh on Track B (search), and Pushkar Kumar on orchestration and integration, mentored by Potluri Krishna Priyatham.

Mentor contact ended in early July. The two track assignments were not taken up, and neither teammate appears in the repository commit history. Both tracks were consolidated into a single-person scope, which is why the majority of commits fall within the final days of the schedule.

References

- [1] M. Henderson, S. Shakya, S. Pradhan, T. Cook, *Quantvolutional Neural Networks: Powering Image Recognition with Quantum Circuits*, Quantum Machine Intelligence, vol. 2, no. 1, 2020.
- [2] L. K. Grover, *A fast quantum mechanical algorithm for database search*, Proceedings of the 28th Annual ACM Symposium on Theory of Computing, 1996, pp. 212–219.
- [3] P. Q. Le, F. Dong, K. Hirota, *A flexible representation of quantum images for polynomial preparation, image compression and processing operations*, Quantum Information Processing, vol. 10, no. 1, pp. 63–84, 2011.
- [4] Y. Zhang, K. Lu, Y. Gao, M. Wang, *NEQR: a novel enhanced quantum representation of digital images*, Quantum Information Processing, vol. 12, no. 8, pp. 2833–2860, 2013.
- [5] M. Boyer, G. Brassard, P. Høyer, A. Tapp, *Tight bounds on quantum searching*, Fortschritte der Physik, vol. 46, pp. 493–505, 1998.
- [6] Qiskit Contributors, *Qiskit: An Open-source Framework for Quantum Computing*, 2023.
- [7] R. Smith, *An Overview of the Tesseract OCR Engine*, Proceedings of the Ninth International Conference on Document Analysis and Recognition (ICDAR), 2007, pp. 629–633.
- [8] N. Otsu, *A Threshold Selection Method from Gray-Level Histograms*, IEEE Transactions on Systems, Man, and Cybernetics, vol. 9, no. 1, pp. 62–66, 1979.